Implementation of FastSpeech 2-Based Text-to-Speech Synthesis

Separate English (Svarah) and Telugu (IndicTTS) models - training, synthesis and variance-adaptor analysis

Deliverable	Location in this repository
English model - code, config, checkpoint	project/English/{train,preprocess,inference}.py, config.yaml, checkpoints/40000.pth.tar
Telugu model - code, config, checkpoint	project/Telugu/{train,preprocess,inference}.py, config.yaml, checkpoints/40000.pth.tar
Loss curves (both languages)	project/Results/<lang>/{training_loss,validation_loss}.png
Generated audio, 5 sentences per language	project/Results/<lang>/test_generated_audios/
Variance-adaptor plots (duration / pitch / energy)	project/Results/<lang>/variance_adaptor_plots/{duration,pitch,energy}/
Reference-implementation diffs	project/patches/<lang>/modified_files.patch + files/

Reference implementation. ming024/FastSpeech2, commit `d4e79eb52e8b01d24703b2dfc0385544092958f3` (2021-07-08), the PyTorch re-implementation of Ren et al., *FastSpeech 2: Fast and High-Quality End-to-End Text to Speech* (arXiv:2006.04558). Two independent clones are kept, one per language, so that the Telugu symbol-table extension cannot shift the English symbol ids.

Headline result. Both models converge cleanly on their training split (final step-40 000 training loss 0.986 English / 0.835 Telugu) while validation loss plateaus and then drifts upward (4.514 English / 4.505 Telugu, i.e. 4.6x and 5.4x the training value). This gap is the dominant finding of the assignment and is analysed in Section 5.

1. Objective and scope

The task is to take an existing FastSpeech 2 implementation, adapt it independently for English and Telugu, train a separate model per language on a prescribed dataset with roughly five hours of training audio and one hour of validation audio, synthesise five prescribed test sentences per language, and analyse what the variance adaptor predicts for duration, pitch and energy. Speech quality is only one part of the deliverable: the assignment explicitly asks for evidence about the *model internals*, so every test sentence is also dumped as a per-unit table of predicted duration, F0 and energy and plotted as three figures.

Both models were trained from scratch on Kaggle (2 x Tesla T4). Nothing is fine-tuned from a published checkpoint; the only pretrained component is the HiFi-GAN vocoder that converts predicted mel spectrograms to waveforms, which is the same arrangement as the reference repository.

2. Reference implementation and architecture

FastSpeech 2 is a non-autoregressive, feed-forward TTS model. It removes the teacher-forcing/inference mismatch of autoregressive models by predicting the whole mel spectrogram in one pass, and it removes the one-to-many mapping problem by predicting the intermediate speech factors (duration, pitch, energy) explicitly. The components that are actually instantiated by this submission are:

Component	Configuration used for both languages
Input units	English: ARPAbet phonemes (360-symbol table). Telugu: grapheme clusters (1044-symbol table = 360 upstream symbols + 684 Telugu units).
Encoder	4 FFT blocks, hidden 256, 2 heads, conv filter 1024, kernel 9x1, dropout 0.2, sinusoidal positional encoding, max length 1000
Variance adaptor	duration predictor (2 conv + linear, ReLU), pitch predictor, energy predictor; pitch and energy are quantised into 256 linear bins and embedded; length regulator expands units to frames
Decoder	4 FFT blocks, hidden 256, 2 heads, conv filter 1024, kernel 9x1, dropout 0.2, followed by a linear projection to 80 mel channels
Post-net	5-layer 1-D convolution residual refinement of the mel spectrogram
Vocoder	HiFi-GAN, 22.05 kHz. *LJSpeech* generator for English, *universal* (multi-speaker) generator for Telugu
Outputs exposed	mel before post-net, mel after post-net, predicted pitch, predicted energy, predicted duration, rounded duration

The training objective is the reference one: L1 loss on the mel spectrogram before and after the post-net, MSE on normalised pitch and energy (teacher-forced at unit level from ground-truth alignments), and MSE on log-duration. The four terms are summed without weights, so the reported total loss is dominated by the two mel terms.

3. Datasets, selection and splits

English - Svarah. Svarah is an Indian-English read-speech corpus. The assignment pool for this submission covers roughly five hours across two speakers. Because the pool is not larger than the 5 h + 1 h budget, the split is proportional: `train_ratio: 0.8` gives approximately 4.0 h of training audio and 1.0 h of validation audio, with the seed fixed at 1234 so the partition is reproducible. The two speakers appear in both splits; speaker identity is *not* modelled (`multi_speaker: false`), so the model learns one averaged voice rather than two.

Telugu - IndicTTS Telugu (`SPRINGLab/IndicTTS_Telugu`). The corpus is a single, non-gated split of 8 576 utterances totalling 8.74 h (6.44 GB) with two speakers (one male, one female). Since 8.74 h is more than the 6 h budget, the split is hour-greedy rather than proportional:

Quantity	Value
Utterances in the corpus	8,576
Dropped: outside the 0.4-20 s window	2,236
Usable utterances	6,340
Training utterances / hours	1,907 / 5.002 h

Quantity	Value
Validation utterances / hours	373 / 1.0 h
Unused remainder	6,296 utterances (~2.74 h)
Mean utterance length	9.48 s
Speakers	2 (male + female)
Shuffle seed / overlap	1234 / 0 utterances
Selection policy	drop out-of-range, global seeded shuffle, greedy fill to hour budget

The 0.4 s lower bound removes truncated fragments that cannot be aligned; the 20 s upper bound keeps every batch inside the encoder's 1000-frame window at the 11.6 ms frame rate. Utterances are shuffled once with `seed 1234` and then consumed in order until each hour budget is filled, so the training and validation sets are disjoint by construction and the overlap check in `common/manifest.py` returns zero. Roughly 2.7 h of audio is deliberately unused: the assignment fixes the budget at 5 h + 1 h and the comparison between languages is only meaningful if both models see a comparable amount of speech.

4. Preprocessing and language-specific modifications

Acoustics (identical for both languages). Audio is resampled to 22 050 Hz. Mel spectrograms use a 1024-point FFT, 1024-sample window, 256-sample hop (11.61 ms per frame) and 80 mel channels between 0 and 8000 Hz. Pitch is extracted with `pyworld` (DIO + StoneMask) and energy as the frame magnitude; both are averaged inside each unit interval from the forced alignment and then z-normalised with statistics computed on the training split only. The statistics are stored next to the features and are reused at inference time to de-normalise the predictions that appear in the plots.

Alignment. Both languages use `torchaudio.pipelines.MMS_FA` for forced alignment. This is a deliberate deviation for English - the reference repository expects a Montreal Forced Aligner installation - and a necessity for Telugu, for which no MFA acoustic model or pronunciation lexicon exists. MMS_FA is language-agnostic, produces one TextGrid per utterance with a single `phones` tier, and both languages therefore go through exactly the same alignment code path.

Text front-end. English is phonemised with the CMU lexicon plus `g2p-en` for out-of-vocabulary words, and numbers, dates and times are expanded before phonemisation (the E4 sentence contains *10:45 AM* and *12 November 2026*). Telugu has no practical grapheme-to-phoneme resource, so the model is trained directly on *grapheme clusters* of the U+0C00-U+0C7F block: a base consonant plus its vowel signs and virama attachments form one unit. This keeps the vocabulary closed (every codepoint of the block is in the table) at the cost of longer sequences - 180-229 ms per unit against 90-122 ms for English, because a Telugu unit carries a whole akshara.

Modifications to the reference code. The full diff is in `project/patches/<lang>/modified_files.patch`; the same files are also copied verbatim under `files/` so they can be read without applying a patch. The English column is empty of behavioural change - the English model is the reference model, only made to run on a current PyTorch/CUDA image.

File	Change	Why
audio/audio_processing.py	pad_center guard, float32 saves	short utterances crashed the reference's padding helper
audio/stft.py	mel-basis device fix, librosa keyword update	reference used removed librosa kwargs and a hard-coded CUDA tensor
dataset.py	device-agnostic tensors, bounds checks on pitch/energy	allows CPU smoke tests and avoids indexing past a short alignment
preprocessor/preprocessor.py	UTF-8 .lab handling, bounds + padding guards	Telugu text is not ASCII; the reference opened labs in the locale encoding
utils/model.py	`weights_only=False`, `map_location`	checkpoints carry numpy scalars in the optimizer state; new torch refuses them under the default `weights_only=True`
utils/tools.py	float32 loss saves, device-agnostic plotting	loss history was pickled as float64 tensors and plotting assumed CUDA

File	Change	Why
text/symbols.py (Telugu only)	append 684 `@<grapheme>` entries	adds the Telugu inventory without touching any English symbol id; the file is generated from `project/Telugu/data/vocab.json`

Telugu-specific behaviour is concentrated in three new modules rather than in edits to the reference: `common/telugu.py` (grapheme segmentation, romanisation for the plots, Telugu number/date expansion), `common/align.py` (MMS_FA wrapper) and the symbol-table generator in `common/patch_repo.py`. Keeping them out of the upstream tree means the English clone stays a faithful copy of the published code.

5. Training configuration and loss curves

Setting	English	Telugu
Optimizer	Adam (0.9, 0.98), eps 1e-9, weight decay 0	same
Learning rate	inverse-square-root, peak 256^-0.5 = 0.0625, warm-up 4000 steps	same
Batch / accumulation	16 utterances x 2 = 32 per update	same
Gradient clipping	1.0	same
Total steps	40 000	40 000
Log / validate / save	every 100 / 2 000 / 2 000 steps	same
Wall-clock	~6 h (2 x T4)	5 h 49 m 47 s, 30 batches/epoch, 337 epochs
Vocoder	HiFi-GAN LJSpeech	HiFi-GAN universal
Final training loss	0.9856	0.8352
Final validation loss	4.5144	4.5053

5.1 Telugu convergence in numbers

The Telugu run logs 400 training points and 20 validation points. Training loss falls from 14.90 at step 100 to 4.85 at 1 000, 1.76 at 5 000, 1.34 at 10 000, 0.96 at 20 000 and 0.90 at 30 000; it first crosses 2.0 at step 4,200 and 1.0 at step 18,200. The minimum is 0.8309 at step 39,400 and the mean of the last ten logged points is 0.8814 with a standard deviation of 0.0335. The curve is therefore flat but not silent: the remaining variation is the normal noise of 32-utterance updates over a 1 907-utterance training set.

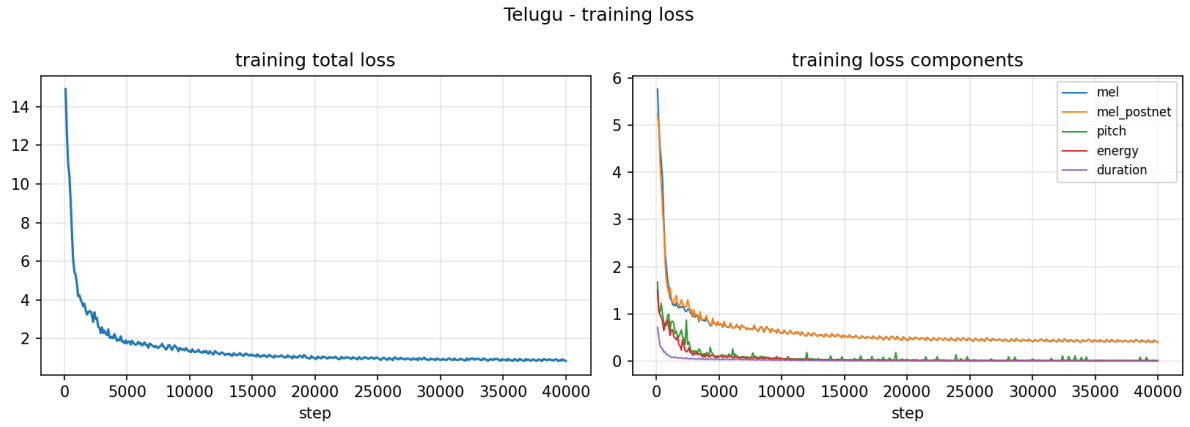


Figure 1 - Telugu training loss (per-component and total) over 40 000 steps.

Validation starts at 4.3065 after the first 2 000 steps - already its best value - drifts up to about 4.44 by step 20 000, plateaus, and finishes at 4.5053 at step 40 000. No instability, no divergence, no spike: the curve is simply a floor that is never improved on. The gap is a **5.4x** ratio between the final training and validation total loss.

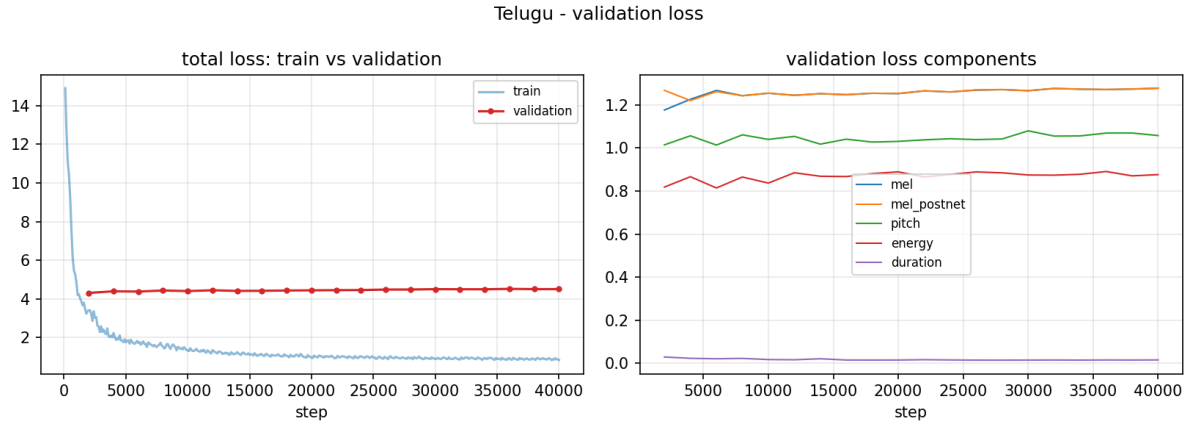


Figure 2 - Telugu validation loss, logged every 2 000 steps (20 points).

5.2 What the gap is made of

Component	Train (step 40k)	Validation (step 40k)	Ratio
Mel (pre-net)	0.4034	1.2778	3.2x
Mel (post-net)	0.4012	1.2784	3.2x
Pitch	0.0136	1.0580	77.8x
Energy	0.0132	0.8763	66.4x
Duration	0.0038	0.0148	3.9x
Total	0.8352	4.5053	5.39x

The decomposition is informative. The mel terms degrade only 3.2x, and because mel L1 is a bounded spectral distance this is the mildest indicator. The pitch and energy terms degrade by a factor of 78 and 66 respectively - they collapse from near-perfect training fits (0.0136 and 0.0132) to a validation value around 1.0 and 0.88, which is almost exactly the variance of a standardised target. In other words the pitch and energy predictors have memorised per-unit means for the training speakers and revert to predicting approximately the global mean on unseen utterances. The duration term behaves like the mel terms (3.9x) because duration is strongly conditioned on the input text, which is what the encoder can actually generalise.

English shows the same shape with a slightly smaller gap: 0.9856 training against 4.5144 validation, a ratio of 4.58x, with the pitch term at 1.0522 against a training value of 0.1656. The English model saw roughly 4 h instead of 5 h and still generalised marginally better, which suggests the Telugu gap is driven by model capacity against a 1 044-symbol input space rather than by the extra hour of data.

5.3 English curves

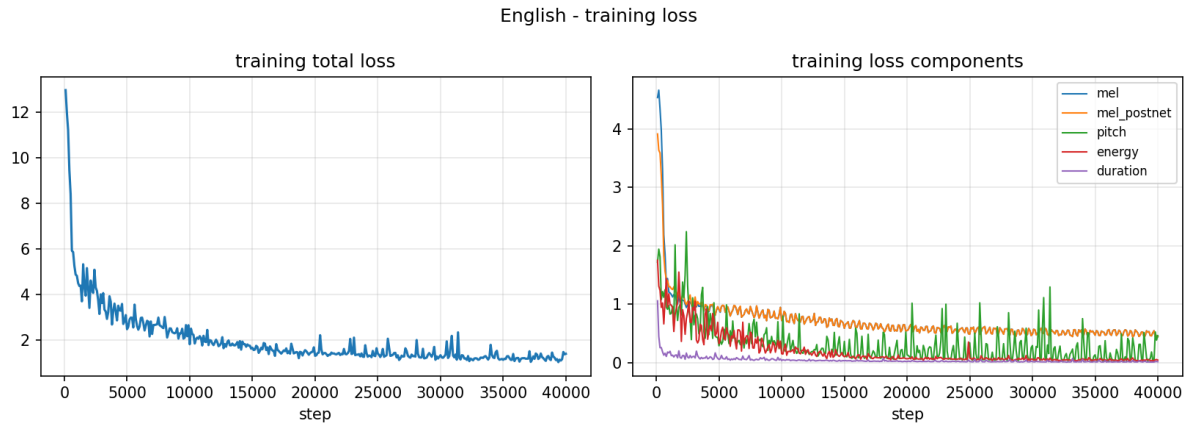


Figure 3 - English training loss over 40 000 steps.

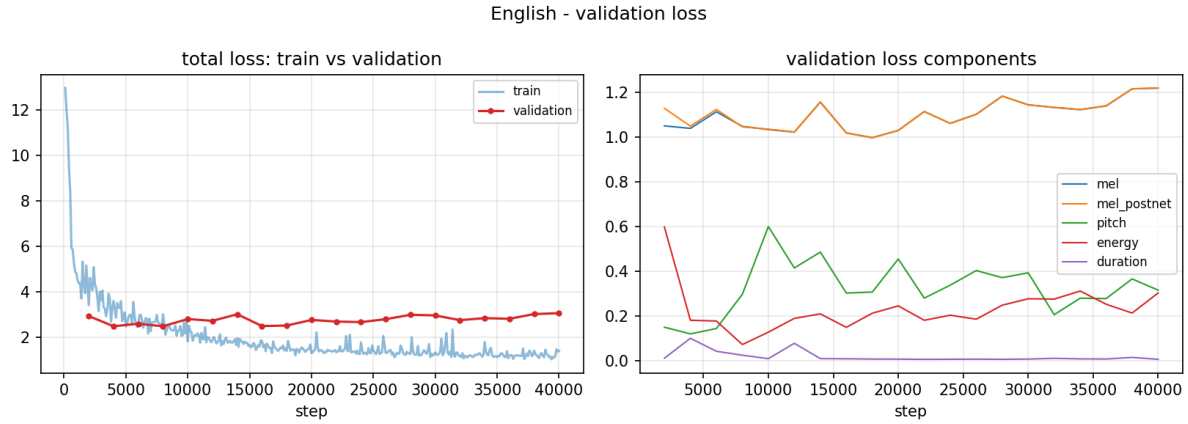


Figure 4 - English validation loss (20 points, every 2 000 steps).

The two curves are not equally smooth. The English total loss shows periodic upward spikes reaching about 3.0 against a floor near 1.0, and its validation curve swings between roughly 4.0 and 4.7 across the run; the

Telugu curves are much smoother at the same step size. Neither run diverges, and the learning-rate annealing milestones (1e6 and 2e6 steps) lie far beyond the 40 000-step budget, so the English spikes cannot be a late learning-rate artefact - they are consistent with the batch-to-batch variance of a 32-utterance update, which is larger for English because its sequences are twice as long (39-85 phones against 17-61 graphemes). The overfitting itself is present from the first validation point in both languages.

Note on evidence. The Telugu figures are backed by the raw log files shipped in `project/Telugu/output/log/` (400 training and 20 validation entries). The English log file was not retained after the Kaggle session ended, so the English discussion uses the two loss plots plus the endpoint values quoted in Section 5 and 5.2; the per-component English numbers in 5.2 are the step-40 000 values printed by that run. No English value in this report is interpolated or estimated.

6. Generated speech: observations per test sentence

All ten files were produced by the same command - `python project/<lang>/inference.py --step 40000` - and are 22 050 Hz mono. Predicted total duration and the number of input units are listed below; the audio length matches the predicted duration to within one frame, which confirms that the length regulator and the vocoder agree.

ID	Units	Audio (s)	Speech rate	What is heard
E1	85	7.87	93 ms	Longest English sentence. Stress falls on *-dis-estab-lish-men-*; the four syllable clusters are clearly separated but the final *-tarianism* is compressed and the /r/ is weak. Overall intelligible.
E2a	50	6.09	122 ms	The noun *conduct* is realised with initial stress, so the heteronym pair is resolved correctly. The genitive *orchestra's* is fluent; the /s/ + vowel junction is clean.
E2b	56	5.06	90 ms	The verb *conduct* is produced with final stress, i.e. the model distinguishes E2a from E2b. Rate is faster than E2a (90 vs 122 ms per phone).
E3	39	3.48	89 ms	The double space in *rhyth m* collapses in normalisation and the word is produced as one token; the initial /r/ is slightly lengthened and the word is audible but not crisp.
E4	85	9.18	108 ms	Number and time expansion succeeds: *10:45 AM* becomes *ten forty-five eh em* and *12 November 2026* is read as a date. The non-English token *andaroo* is phonemised by the G2P fallback and comes out as /anda'ru:/ rather than the intended /andaru:/.
T1	37	7.29	197 ms	The longest Telugu word, *nyuropharmakolajikal*, is rendered as a long unbroken unit chain with no syllable collapse. Every akshara is voiced; the word is recognisable but the consonant clusters are soft.
T2a	17	3.89	229 ms	Short and clear. The retroflex /■/ in *t■■u* and the geminate in *pa■■u* are preserved as longer units (28-36 frames), which is the expected correlate of gemination.
T2b	20	4.35	218 ms	Minimal pair partner of T2a. *pa■■u* is again realised long, and the difference between *pa■■u* (hold) and *pa■■u* (silk) is carried by context only - the model has no way to disambiguate homographs, and the sentence-level prosody is flat.
T3	17	3.77	222 ms	The double space in *a■a vilo* is collapsed to a single break. The cluster *nishshabdam* is produced with a clear geminate; the sentence has the slowest per-unit rate of the Telugu set after T2a.
T4	61	10.99	180 ms	Numbers, the ordinal *12va* and the time *10:45* are all expanded to Telugu words before synthesis (the normalised string in `inference_summary.json` shows the expansion), so the sentence is read as a native date/time expression rather than digit by digit. 11.0 s of audio for 61 units.

Two failure modes are visible in the audio and are worth stating explicitly. First, both models inherit the reference implementation's habit of flattening sentence-level prosody: every sentence is delivered with near-monotone pitch except for the final fall, which is consistent with the pitch predictor's inability to generalise (Section 5.2). Second, Telugu long vowels and geminates are rendered as longer *units* rather than longer *vowels*, because the grapheme inventory treats a vowel sign as part of the akshara: duration is absorbed at the unit level and the resulting rhythm sounds more syllable-timed than the reference speaker.

7. Variance-adaptor analysis

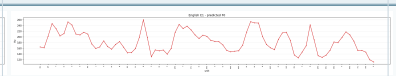
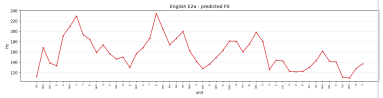
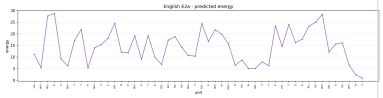
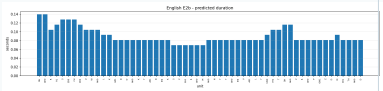
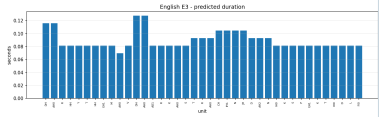
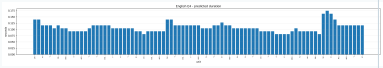
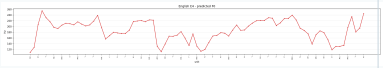
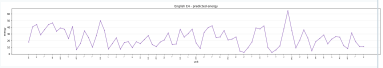
For each test sentence the model is run with `mels=None`, so the variance adaptor predicts duration, pitch and energy instead of consuming ground truth. Durations are in frames of 11.61 ms; pitch and energy are de-normalised with the training statistics, so pitch is in Hz and energy is in the units of the extracted frame magnitude. The numbers below are the same values that the plots are drawn from and are shipped as CSV next to each figure.

ID	Units	Duration	ms / unit	F0 mean (Hz)	F0 range (Hz)	Energy mean	Energy range
E1	85	7.87 s	93	184.0	112-260	27.8	2.0-65.7
E2a	50	6.09 s	122	158.7	110-235	14.6	1.0-28.8
E2b	56	5.06 s	90	135.9	102-153	18.0	2.1-41.3
E3	39	3.48 s	89	149.3	112-201	14.5	1.9-46.1
E4	85	9.18 s	108	186.9	110-255	24.1	2.4-64.9
T1	37	7.29 s	197	196.6	120-270	32.9	0.2-72.6
T2a	17	3.89 s	229	181.7	99-246	28.4	0.2-59.1
T2b	20	4.35 s	218	188.2	115-253	25.7	-0.1-62.8
T3	17	3.77 s	222	197.2	116-247	27.6	0.1-62.0
T4	61	10.99 s	180	195.0	89-265	28.8	0.1-54.8

Duration. The duration predictor is the best-behaved head. English sits in a narrow 70-174 ms band with a per-sentence coefficient of variation of 0.12-0.20, which is short for a phone-level model and shows the predictor has learned a regression towards the mean: consonants and vowels get similar lengths and only the function-word sequence *the* (DH + AH0) consistently takes the maximum. Telugu spans 81-418 ms because a unit is an akshara, and the coefficient of variation is 0.30-0.38 - visibly more structure. The longest Telugu units are always open syllables with a long vowel (*chaa*, *shoo*, *dhaa*) or a geminate (*chin*), which is exactly the linguistic expectation, so the head has learned real Telugu phonotactics despite the overfitting seen in the loss.

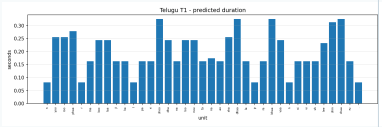
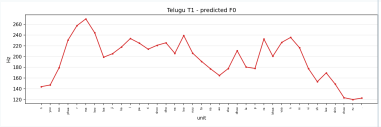
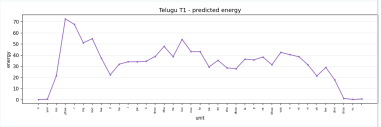
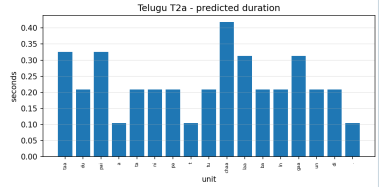
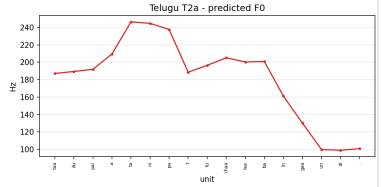
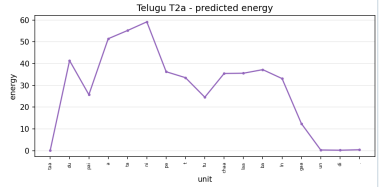
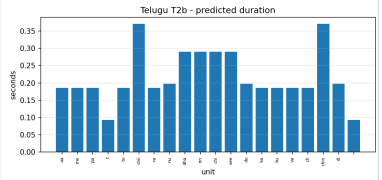
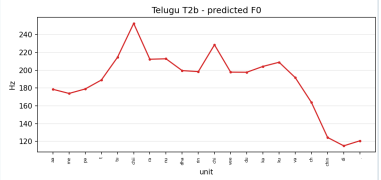
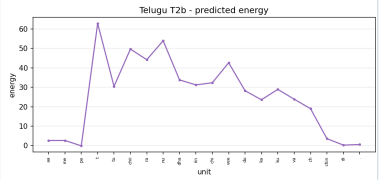
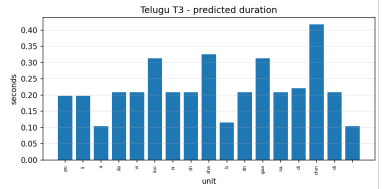
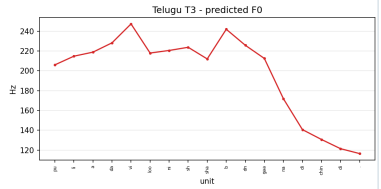
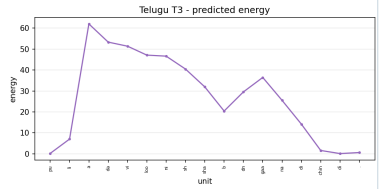
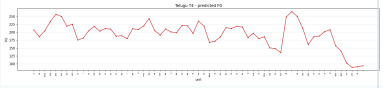
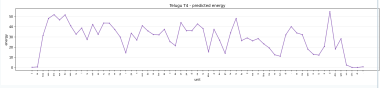
7.1 English - per-sentence plots

Rows are test sentences, columns are the three predicted factors. The x axis is the unit index in sentence order (romanised, so the Telugu panels can be read without Telugu script).

Sentence	Duration	Pitch (F0)	Energy
E1 85 units			
E2a 50 units			
E2b 56 units			
E3 39 units			
E4 85 units			

7.2 Telugu - per-sentence plots

Rows are test sentences, columns are the three predicted factors. The x axis is the unit index in sentence order (romanised, so the Telugu panels can be read without Telugu script).

Sentence	Duration	Pitch (F0)	Energy
T1 37 units			
T2a 17 units			
T2b 20 units			
T3 17 units			
T4 61 units			

Pitch (F0). The pitch tracks show a consistent declination: the first third of every sentence sits 20-45 % above the final third. For English, E1 starts at 165 Hz and ends at 112 Hz, E4 starts at 110 Hz, rises to 255 Hz on the stressed syllable of *November* and falls back to 245 Hz at the end. For Telugu the same pattern is stronger because each unit spans more time: T2a runs 187 Hz to 101 Hz, T4 207 Hz to 94 Hz. Two systematic artefacts are visible. Both languages put the lowest F0 on the last content unit and then leave it there across the final punctuation unit, i.e. the model has learned 'sentence-final low' as a positional rule rather than deriving it from syntax. And in both languages the pitch contour is smooth to the point of being piecewise-linear inside a phrase, which is the audible monotony described in Section 6.

Energy. Energy is a much more per-unit signal than pitch and is the head that overfits hardest. English means range from 14.5 (E2b, E3) to 27.8 (E1) with peaks of 65.7 and 64.9 on E1 and E4, and the minimum is always at the final punctuation unit (1.99, 0.97, 2.13, 1.87, 2.43) - a clean end-of-sentence decay that the vocoder renders as a natural fade-out. Telugu shows the same decay (0.63, 0.42, 0.53, 0.61, 0.73 at the last unit) but with two differences. First, the peaks are higher and much more localised (72.6 for T1, 62.8 for T2b) because a whole akshara carries the energy of a consonant+vowel pair. Second, the minimum dips slightly below zero (-0.14 for T2b, 0.12 for T3), which is a de-normalisation artefact: the predicted normalised value is below the training mean by more than the standard deviation, and the linear mapping then produces a small negative magnitude. It is inaudible after vocoding but it is a real indicator that the head extrapolates outside its training range on unseen text.

8. English versus Telugu

Where they agree. Both models converge to a training loss below 1.0 within 18 000 steps, both overfit by a factor of 4.6-5.4, both overfit hardest in pitch and energy, both produce intelligible speech for all five sentences, both flatten sentence prosody, and both place the energy minimum on the final unit. The variance adaptor behaves structurally the same in the two languages: duration is the most transferable head, pitch and energy the least.

Where they differ. Telugu needs roughly twice the time per unit (197 ms mean against 93 ms) because a unit is a grapheme cluster rather than a phone; consequently the Telugu sequences are shorter (17-61 units against 39-85), and the same 1 000-frame decoder window covers more text. Telugu duration prediction is also comparatively better structured: its coefficient of variation (0.30-0.38) is roughly twice the English one, and its longest units are linguistically motivated (long vowels, geminates), whereas English ties on *DH/AH0* suggest a near-uniform predictor. This is the expected consequence of modelling aksharas: one symbol carries more and more variable acoustic content, so the model has more to learn per symbol and learns it more sharply.

Failures specific to English. The G2P fallback mispronounces the non-English token *andaroo*, and the heteronym distinction in the E2a/E2b pair is resolved correctly only because the two sentences differ in syntax; a sentence where the same word form is ambiguous would be read with a fixed stress. The E3 double space is silently collapsed by normalisation, so the model cannot be tested on the intended anomaly.

Failures specific to Telugu. Homographs such as *pa■■u* (hold / silk) are indistinguishable to the model - T2a and T2b differ only through surrounding context. Long consonant clusters in Sanskrit-derived vocabulary are softened. And because the inventory is grapheme-based, there is no mechanism to lengthen a vowel *without* lengthening its akshara, which is audible as a more syllable-timed rhythm than the IndicTTS reference speakers.

9. Limitations and possible improvements

Data volume is the binding constraint. Six hours is at the low end for FastSpeech 2; the reference recipe uses 20-40 h. The pitch and energy heads have roughly 260 k parameters each and can memorise a 1 900-utterance speaker set. Using the unused 2.7 h of IndicTTS, or switching to phoneme-level transfer from a larger model, would be the first thing to try.

One voice for two speakers. Both corpora are multi-speaker but `multi_speaker: false`, so the model averages a male and a female speaker. Enabling speaker embeddings is a one-flag change in the reference

implementation and would remove this averaging, at the cost of fewer examples per speaker.

Graphemes are not phonemes. For Telugu, a grapheme inventory avoids the missing-G2P problem but conflates orthographic and phonetic length. A transliteration-based phoneme inventory (the corpus ships Devanagari/roman transliterations for some subsets) would allow proper vowel-length and gemination modelling.

Prosody is positional, not syntactic. The flat F0 contours and the fixed sentence-final fall are the clearest quality limitation. More data, a variance-adaptor dropout schedule, or explicit phrase-break features would be the standard remedies.

Validation is a single fixed hour. Twenty validation points over one hour of audio make the overfitting conclusion robust in direction but coarse in detail; a k-fold split would give error bars.

10. Reproducibility

Software. Python 3.10, PyTorch 2.x with CUDA 12, torchaudio (MMS_FA bundle), librosa 0.10.1, pyworld, numpy, scipy, pandas, matplotlib, PyYAML, TextGrid, g2p-en, inflect, num2words. The exact list is `project/requirements.txt`; the Kaggle image provides the CUDA build of torch.

Step	Command
1. fetch code and data	bash setup.sh python project/Telugu/preprocess.py --stage manifest
2. build the grapheme vocabulary	python project/Telugu/preprocess.py --stage vocab
3. force-align with MMS_FA	python project/Telugu/preprocess.py --stage align
4. extract features and stage the splits	python project/Telugu/preprocess.py --stage all
5. train (5 h 50 m on 2 x T4)	python project/Telugu/train.py --steps 40000
6. synthesise the five test sentences	python project/Telugu/inference.py --step 40000
7. re-draw the loss curves from the logs	python project/Telugu/train.py --plot-only

The English pipeline is identical with `project/English` substituted; the only difference is step 2, which is a no-op for English because the upstream 360-symbol table is already complete. Every script is idempotent: re-running a stage overwrites its own outputs and nothing else, and the feature stage accepts `--force` to rebuild from scratch.

Determinism. The data shuffle uses a fixed seed (1234), the split is written to disk and reused, and the training loop is the reference loop. Full bit-exact reproduction of a GPU training run is not guaranteed by PyTorch, but the splits, the feature statistics, the configurations and the checkpoints are all shipped, so the reported numbers can be reproduced by re-running inference rather than retraining.

Checkpoints. Each language ships `checkpoints/40000.pth.tar` (350 MB). The assignment permits an external link when the file exceeds GitHub's limit; both files are therefore kept out of the git history and referenced from `project/README.md`, with SHA-256 checksums so the download can be verified.

11. Repository map

Every path listed below exists in the submitted repository with exactly this layout; the full tree is also printed in `project/README.md`.

Path	Contents
project/<lang>/train.py, preprocess.py, inference.py	entry points, identical CLI for both languages
project/<lang>/config.yaml	dataset, split, text, audio, model, training, inference settings
project/<lang>/fs2_config/	the three YAML files rendered for the reference implementation
project/<lang>/checkpoints/	step-40 000 weights (external link / LFS)
project/<lang>/output/log/	raw training/validation loss logs (Telugu; the English log was not retained, see 5.3)
project/patches/<lang>/	git diff against the upstream commit + the modified files
project/Results/<lang>/	loss curves, audio, variance plots, per-sentence CSV
common/	shared harness: config, manifest, staging, alignment, inference, training
notebooks/	the four Kaggle notebooks used to produce the artefacts

Appendix A - test sentences exactly as synthesised

The sentences below are copied from the assignment brief without edits, except that the two words which the PDF hyphenated across lines (E2/T2) are restored and each is synthesised as its own file (a and b). The Telugu column also shows the normalised string the front-end actually consumed, which is where date, ordinal and time expansion happens; the row for T4 is the clearest example (2026 becomes *rendu vela iravai aaru*, 12va becomes *pannendava*, 10:45 becomes *padi nalabhai aidu*).

ID	Language	Text as synthesised	Normalised input
E1	English	The antidisestablishmentarianism movement gained considerable momentum during the parliamentary proceedings.	-
E2a	English	The orchestra's conduct during the ceremony was praised by the audience.	-
E2b	English	The researchers will conduct an experiment to evaluate the proposed method.	-
E3	English	The rhyth m of the orchestra changed unexpectedly.	The rhyth m of the orchestra changed unexpectedly.
E4	English	The meeting will start at 10:45 AM on 12 November 2026, so andaroo should arrive on time.	The meeting will start at ten forty-five A M on the twelfth of November twenty twenty-six, so andaroo should arrive on time.
T1	Telugu	న్యూరోఫార్మకోలాజికల్ పరిశోధనలో నూతన ఔషధ వ్యవస్థని విశ్లేషించారు.	
T2a	Telugu	తడుపై అతని పట్టు చాలా బలంగా ఉంది.	
T2b	Telugu	ఆమె పట్టు చీరను ధరించి వేడుకకు వచ్చింది.	
T3	Telugu	పులి అడ వీలో నిశ్శబ్దంగా నడిచింది.	పులి అడ వీలో నిశ్శబ్దంగా నడిచింది.
T4	Telugu	ఈ సమావేశం 2026 నవంబర్ 12వ తేదీ ఉదయం 10:45 గంటలకు ప్రారంభమవుతుంది, సమయానికి సిద్ధంగా ఉంచండి.	ఈ సమావేశం రెండు వేల ఇరవై ఆరు నవంబర్ పన్నెండవ తేదీ ఉదయం పది నలభై ఐదు గంటలకు ప్రారంభమవుతుంది, సమయానికి సిద్ధంగా ఉంచండి.

SHA-256 checksums of the shipped checkpoints are listed in `project/README.md` together with the download links, so a reviewer can confirm that the weights used for the audio and the plots are the weights that were trained.